MLA02 – FUNDAMENTALS OF MACHINE LEARNING

LAB EXPERIMENT 3

Demonstrate the working of the Decision Tree based ID3 algorithm.

Five Independent Problems with Aim, Procedure, Python Code and Result

Name: S. MOHAN KUMAR

Register Number: 1925260331

Each program is self-contained and uses only Python's standard library.

Problem 1: ID3 Decision Tree — Play Tennis

AIM

To implement the ID3 decision tree algorithm using entropy and information gain, then classify whether tennis can be played under a new weather condition.

PROCEDURE

1. Import math and Counter for entropy and class-frequency calculations.
2. Store the Play Tennis training samples as categorical records.
3. Calculate dataset entropy and information gain for each unused attribute.
4. Choose the attribute with the highest information gain and split the data.
5. Repeat recursively until a node contains one class or no useful attribute remains.
6. Classify the new sample and display the predicted class.

PYTHON CODING

Copy and run the complete program below in Python 3.

```
import math
from collections import Counter

def entropy(rows, target):
    counts = Counter(row[target] for row in rows)
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())

def information_gain(rows, feature, target):
    total_entropy = entropy(rows, target)
    values = set(row[feature] for row in rows)
    remainder = 0.0
    for value in values:
        subset = [row for row in rows if row[feature] == value]
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def majority_class(rows, target):
    return Counter(row[target] for row in rows).most_common(1)[0][0]

def build_tree(rows, features, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not features:
        return majority_class(rows, target)

    gains = [(information_gain(rows, feature, target), feature)
             for feature in features]
    # Stable tie-breaking: keep the first feature in the feature list.
    best_gain, best_feature = max(gains, key=lambda item: item[0])
    if best_gain <= 1e-12:
        return majority_class(rows, target)

    tree = {"feature": best_feature, "default": majority_class(rows, target),
            "branches": {}}
    remaining = [feature for feature in features if feature != best_feature]
    for value in sorted(set(row[best_feature] for row in rows)):
        subset = [row for row in rows if row[best_feature] == value]
        tree["branches"][value] = build_tree(subset, remaining, target)
    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    feature_value = sample.get(tree["feature"])
    branch = tree["branches"].get(feature_value)
    if branch is None:
        return tree["default"]
    return predict(branch, sample)

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + str(tree))
        return
    print(indent + str(tree["feature"]))
    for value, subtree in tree["branches"].items():
        print(indent + "  [" + str(value) + "]")
        print_tree(subtree, indent + "    ")


data = [
    {"Outlook":"Sunny", "Temperature":"Hot", "Humidity":"High",
     "Wind":"Weak", "Play":"No"},
```

```
    {"Outlook":"Sunny", "Temperature":"Hot", "Humidity":"High",
     "Wind":"Strong", "Play":"No"},
    {"Outlook":"Overcast", "Temperature":"Hot", "Humidity":"High",
     "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Temperature":"Mild", "Humidity":"High",
     "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Temperature":"Cool", "Humidity":"Normal",
     "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Temperature":"Cool", "Humidity":"Normal",
     "Wind":"Strong", "Play":"No"},
    {"Outlook":"Overcast", "Temperature":"Cool", "Humidity":"Normal",
     "Wind":"Strong", "Play":"Yes"},
    {"Outlook":"Sunny", "Temperature":"Mild", "Humidity":"High",
     "Wind":"Weak", "Play":"No"},
    {"Outlook":"Sunny", "Temperature":"Cool", "Humidity":"Normal",
     "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Temperature":"Mild", "Humidity":"Normal",
     "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Sunny", "Temperature":"Mild", "Humidity":"Normal",
     "Wind":"Strong", "Play":"Yes"},
    {"Outlook":"Overcast", "Temperature":"Mild", "Humidity":"High",
     "Wind":"Strong", "Play":"Yes"},
    {"Outlook":"Overcast", "Temperature":"Hot", "Humidity":"Normal",
     "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Temperature":"Mild", "Humidity":"High",
     "Wind":"Strong", "Play":"No"},
]
target = "Play"
sample = {"Outlook":"Sunny", "Temperature":"Cool",
          "Humidity":"High", "Wind":"Strong"}

features = [key for key in data[0] if key != target]
tree = build_tree(data, features, target)

print("Decision Tree:")
print_tree(tree)
print("New sample:", sample)
print("Predicted class:", predict(tree, sample))
```

SAMPLE INPUT

The training dataset and test sample are included in the code above.

SAMPLE OUTPUT

```
Decision Tree:
Outlook
  [Overcast]
    -> Yes
  [Rain]
    Wind
      [Strong]
        -> No
      [Weak]
        -> Yes
  [Sunny]
    Humidity
      [High]
        -> No
      [Normal]
        -> Yes
New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Predicted class: No
```

RESULT

The ID3 algorithm builds a decision tree from the Play Tennis data and predicts No for the supplied sample.

Problem 2: ID3 — Weather-Based Outdoor Activity

AIM

To build an ID3 decision tree from weather observations and predict whether an outdoor activity is suitable for a new weather sample.

PROCEDURE

1. Create a categorical dataset with sky, temperature, humidity, and wind conditions.
2. Define entropy to measure impurity in the target labels.
3. Compute information gain for every available feature.
4. Select the feature with maximum gain and recursively build the tree.
5. Stop splitting when all labels are equal or no features remain.
6. Use the constructed tree to predict the activity decision for a new sample.

PYTHON CODING

Copy and run the complete program below in Python 3.

```
import math
from collections import Counter

def entropy(rows, target):
    counts = Counter(row[target] for row in rows)
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())

def information_gain(rows, feature, target):
    total_entropy = entropy(rows, target)
    values = set(row[feature] for row in rows)
    remainder = 0.0
    for value in values:
        subset = [row for row in rows if row[feature] == value]
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def majority_class(rows, target):
    return Counter(row[target] for row in rows).most_common(1)[0][0]

def build_tree(rows, features, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not features:
        return majority_class(rows, target)

    gains = [(information_gain(rows, feature, target), feature)
             for feature in features]
    # Stable tie-breaking: keep the first feature in the feature list.
    best_gain, best_feature = max(gains, key=lambda item: item[0])
    if best_gain <= 1e-12:
        return majority_class(rows, target)

    tree = {"feature": best_feature, "default": majority_class(rows, target),
            "branches": {}}
    remaining = [feature for feature in features if feature != best_feature]
    for value in sorted(set(row[best_feature] for row in rows)):
        subset = [row for row in rows if row[best_feature] == value]
        tree["branches"][value] = build_tree(subset, remaining, target)
    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    feature_value = sample.get(tree["feature"])
    branch = tree["branches"].get(feature_value)
    if branch is None:
        return tree["default"]
    return predict(branch, sample)

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + str(tree))
        return
    print(indent + str(tree["feature"]))
    for value, subtree in tree["branches"].items():
        print(indent + "  [" + str(value) + "]")
        print_tree(subtree, indent + "    ")


data = [
    {"Sky":"Clear", "Temp":"Warm", "Humidity":"High",
     "Wind":"Weak", "Activity":"No"},
```

```
    {"Sky":"Clear", "Temp":"Warm", "Humidity":"High",
     "Wind":"Strong", "Activity":"No"},
    {"Sky":"Cloudy", "Temp":"Warm", "Humidity":"High",
     "Wind":"Weak", "Activity":"Yes"},
    {"Sky":"Rainy", "Temp":"Mild", "Humidity":"High",
     "Wind":"Weak", "Activity":"Yes"},
    {"Sky":"Rainy", "Temp":"Cool", "Humidity":"Normal",
     "Wind":"Weak", "Activity":"Yes"},
    {"Sky":"Rainy", "Temp":"Cool", "Humidity":"Normal",
     "Wind":"Strong", "Activity":"No"},
    {"Sky":"Cloudy", "Temp":"Cool", "Humidity":"Normal",
     "Wind":"Strong", "Activity":"Yes"},
    {"Sky":"Clear", "Temp":"Mild", "Humidity":"High",
     "Wind":"Weak", "Activity":"No"},
    {"Sky":"Clear", "Temp":"Cool", "Humidity":"Normal",
     "Wind":"Weak", "Activity":"Yes"},
    {"Sky":"Rainy", "Temp":"Mild", "Humidity":"Normal",
     "Wind":"Weak", "Activity":"Yes"},
    {"Sky":"Clear", "Temp":"Mild", "Humidity":"Normal",
     "Wind":"Strong", "Activity":"Yes"},
    {"Sky":"Cloudy", "Temp":"Mild", "Humidity":"High",
     "Wind":"Strong", "Activity":"Yes"},
    {"Sky":"Cloudy", "Temp":"Warm", "Humidity":"Normal",
     "Wind":"Weak", "Activity":"Yes"},
    {"Sky":"Rainy", "Temp":"Mild", "Humidity":"High",
     "Wind":"Strong", "Activity":"No"},
]
target = "Activity"
sample = {"Sky":"Clear", "Temp":"Cool",
          "Humidity":"High", "Wind":"Weak"}

features = [key for key in data[0] if key != target]
tree = build_tree(data, features, target)

print("Decision Tree:")
print_tree(tree)
print("New sample:", sample)
print("Predicted class:", predict(tree, sample))
```

SAMPLE INPUT

The training dataset and test sample are included in the code above.

SAMPLE OUTPUT

```
Decision Tree:
Sky
  [Clear]
    Humidity
      [High]
        -> No
      [Normal]
        -> Yes
  [Cloudy]
    -> Yes
  [Rainy]
    Wind
      [Strong]
        -> No
      [Weak]
        -> Yes
New sample: {'Sky': 'Clear', 'Temp': 'Cool', 'Humidity': 'High', 'Wind': 'Weak'}
Predicted class: No
```

RESULT

The ID3 algorithm builds a weather-based tree and predicts No for the supplied sample.

Problem 3: ID3 — Student Pass Prediction

AIM

To implement ID3 for a small student dataset and predict a student's result using attendance, assignment completion, and internal-test performance.

PROCEDURE

1. Represent each student's attributes and Pass/Fail label as a dictionary.
2. Calculate the entropy of the complete training dataset.
3. Find information gain for each available attribute.
4. Split on the attribute with the highest information gain.
5. Continue splitting recursively to create the decision tree.
6. Predict and print the result for a new student's profile.

PYTHON CODING

Copy and run the complete program below in Python 3.

```
import math
from collections import Counter

def entropy(rows, target):
    counts = Counter(row[target] for row in rows)
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())

def information_gain(rows, feature, target):
    total_entropy = entropy(rows, target)
    values = set(row[feature] for row in rows)
    remainder = 0.0
    for value in values:
        subset = [row for row in rows if row[feature] == value]
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def majority_class(rows, target):
    return Counter(row[target] for row in rows).most_common(1)[0][0]

def build_tree(rows, features, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not features:
        return majority_class(rows, target)

    gains = [(information_gain(rows, feature, target), feature)
             for feature in features]
    # Stable tie-breaking: keep the first feature in the feature list.
    best_gain, best_feature = max(gains, key=lambda item: item[0])
    if best_gain <= 1e-12:
        return majority_class(rows, target)

    tree = {"feature": best_feature, "default": majority_class(rows, target),
            "branches": {}}
    remaining = [feature for feature in features if feature != best_feature]
    for value in sorted(set(row[best_feature] for row in rows)):
        subset = [row for row in rows if row[best_feature] == value]
        tree["branches"][value] = build_tree(subset, remaining, target)
    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    feature_value = sample.get(tree["feature"])
    branch = tree["branches"].get(feature_value)
    if branch is None:
        return tree["default"]
    return predict(branch, sample)

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + str(tree))
        return
    print(indent + str(tree["feature"]))
    for value, subtree in tree["branches"].items():
        print(indent + "  [" + str(value) + "]")
        print_tree(subtree, indent + "    ")


data = [
    {"Attendance":"High", "Assignment":"Done", "Internal":"Good",
     "Result":"Pass"},
```

```
    {"Attendance":"High", "Assignment":"Done", "Internal":"Poor",
     "Result":"Pass"},
    {"Attendance":"High", "Assignment":"NotDone", "Internal":"Good",
     "Result":"Pass"},
    {"Attendance":"Low", "Assignment":"Done", "Internal":"Good",
     "Result":"Pass"},
    {"Attendance":"Low", "Assignment":"NotDone", "Internal":"Poor",
     "Result":"Fail"},
    {"Attendance":"Low", "Assignment":"NotDone", "Internal":"Good",
     "Result":"Fail"},
    {"Attendance":"High", "Assignment":"NotDone", "Internal":"Poor",
     "Result":"Fail"},
    {"Attendance":"Low", "Assignment":"Done", "Internal":"Poor",
     "Result":"Fail"},
    {"Attendance":"High", "Assignment":"Done", "Internal":"Good",
     "Result":"Pass"},
    {"Attendance":"Low", "Assignment":"Done", "Internal":"Good",
     "Result":"Pass"},
]
target = "Result"
sample = {"Attendance":"High", "Assignment":"Done",
          "Internal":"Poor"}

features = [key for key in data[0] if key != target]
tree = build_tree(data, features, target)

print("Decision Tree:")
print_tree(tree)
print("New sample:", sample)
print("Predicted class:", predict(tree, sample))
```

SAMPLE INPUT

The training dataset and test sample are included in the code above.

SAMPLE OUTPUT

```
Decision Tree:
Assignment
  [Done]
    Internal
      [Good]
        -> Pass
      [Poor]
        Attendance
          [High]
            -> Pass
          [Low]
            -> Fail
  [NotDone]
    Attendance
      [High]
        Internal
          [Good]
            -> Pass
          [Poor]
            -> Fail
      [Low]
        -> Fail
New sample: {'Attendance': 'High', 'Assignment': 'Done', 'Internal': 'Poor'}
Predicted class: Pass
```

RESULT

The ID3 algorithm builds a student-result tree and predicts Pass for the supplied sample.

Problem 4: ID3 — Loan Approval

AIM

To construct a decision tree using ID3 and classify a sample loan application as Approved or Rejected from categorical applicant details.

PROCEDURE

1. Prepare training examples containing income, employment, credit history, and loan decision.
2. Calculate entropy from the Approved/Rejected labels.
3. Calculate information gain for each candidate attribute.
4. Choose the highest-gain attribute as the current decision node.
5. Build branches recursively until a pure class or no features remain.
6. Classify the given loan applicant and display the prediction.

PYTHON CODING

Copy and run the complete program below in Python 3.

```
import math
from collections import Counter

def entropy(rows, target):
    counts = Counter(row[target] for row in rows)
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())

def information_gain(rows, feature, target):
    total_entropy = entropy(rows, target)
    values = set(row[feature] for row in rows)
    remainder = 0.0
    for value in values:
        subset = [row for row in rows if row[feature] == value]
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def majority_class(rows, target):
    return Counter(row[target] for row in rows).most_common(1)[0][0]

def build_tree(rows, features, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not features:
        return majority_class(rows, target)

    gains = [(information_gain(rows, feature, target), feature)
             for feature in features]
    # Stable tie-breaking: keep the first feature in the feature list.
    best_gain, best_feature = max(gains, key=lambda item: item[0])
    if best_gain <= 1e-12:
        return majority_class(rows, target)

    tree = {"feature": best_feature, "default": majority_class(rows, target),
            "branches": {}}
    remaining = [feature for feature in features if feature != best_feature]
    for value in sorted(set(row[best_feature] for row in rows)):
        subset = [row for row in rows if row[best_feature] == value]
        tree["branches"][value] = build_tree(subset, remaining, target)
    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    feature_value = sample.get(tree["feature"])
    branch = tree["branches"].get(feature_value)
    if branch is None:
        return tree["default"]
    return predict(branch, sample)

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + str(tree))
        return
    print(indent + str(tree["feature"]))
    for value, subtree in tree["branches"].items():
        print(indent + "  [" + str(value) + "]")
        print_tree(subtree, indent + "    ")


data = [
    {"Income":"High", "Job":"Stable", "Credit":"Good",
     "Loan":"Approved"},
```

```
    {"Income":"High", "Job":"Stable", "Credit":"Poor",
     "Loan":"Rejected"},
    {"Income":"High", "Job":"Unstable", "Credit":"Good",
     "Loan":"Approved"},
    {"Income":"Low", "Job":"Stable", "Credit":"Good",
     "Loan":"Approved"},
    {"Income":"Low", "Job":"Unstable", "Credit":"Poor",
     "Loan":"Rejected"},
    {"Income":"Low", "Job":"Unstable", "Credit":"Good",
     "Loan":"Rejected"},
    {"Income":"High", "Job":"Unstable", "Credit":"Poor",
     "Loan":"Rejected"},
    {"Income":"Low", "Job":"Stable", "Credit":"Poor",
     "Loan":"Rejected"},
    {"Income":"High", "Job":"Stable", "Credit":"Good",
     "Loan":"Approved"},
    {"Income":"Low", "Job":"Stable", "Credit":"Good",
     "Loan":"Approved"},
]
target = "Loan"
sample = {"Income":"High", "Job":"Stable", "Credit":"Poor"}

features = [key for key in data[0] if key != target]
tree = build_tree(data, features, target)

print("Decision Tree:")
print_tree(tree)
print("New sample:", sample)
print("Predicted class:", predict(tree, sample))
```

SAMPLE INPUT

The training dataset and test sample are included in the code above.

SAMPLE OUTPUT

```
Decision Tree:
Credit
  [Good]
    Job
      [Stable]
        -> Approved
      [Unstable]
        Income
          [High]
            -> Approved
          [Low]
            -> Rejected
  [Poor]
    -> Rejected
New sample: {'Income': 'High', 'Job': 'Stable', 'Credit': 'Poor'}
Predicted class: Rejected
```

RESULT

The ID3 algorithm builds a loan decision tree and predicts Rejected for the supplied sample.

Problem 5: ID3 — Customer Purchase Prediction

AIM

To use the ID3 algorithm to predict whether a customer will purchase a product based on customer type, promotion, and visit frequency.

PROCEDURE

1. Create a labelled dataset with customer and shopping attributes.
2. Compute entropy of the Purchase target variable.
3. Calculate information gain for each unused feature.
4. Select the feature with the largest information gain to split the dataset.
5. Repeat the process recursively until the stopping condition is reached.
6. Apply the tree to a new customer and report the predicted purchase decision.

PYTHON CODING

Copy and run the complete program below in Python 3.

```
import math
from collections import Counter

def entropy(rows, target):
    counts = Counter(row[target] for row in rows)
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())

def information_gain(rows, feature, target):
    total_entropy = entropy(rows, target)
    values = set(row[feature] for row in rows)
    remainder = 0.0
    for value in values:
        subset = [row for row in rows if row[feature] == value]
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def majority_class(rows, target):
    return Counter(row[target] for row in rows).most_common(1)[0][0]

def build_tree(rows, features, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not features:
        return majority_class(rows, target)

    gains = [(information_gain(rows, feature, target), feature)
             for feature in features]
    # Stable tie-breaking: keep the first feature in the feature list.
    best_gain, best_feature = max(gains, key=lambda item: item[0])
    if best_gain <= 1e-12:
        return majority_class(rows, target)

    tree = {"feature": best_feature, "default": majority_class(rows, target),
            "branches": {}}
    remaining = [feature for feature in features if feature != best_feature]
    for value in sorted(set(row[best_feature] for row in rows)):
        subset = [row for row in rows if row[best_feature] == value]
        tree["branches"][value] = build_tree(subset, remaining, target)
    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    feature_value = sample.get(tree["feature"])
    branch = tree["branches"].get(feature_value)
    if branch is None:
        return tree["default"]
    return predict(branch, sample)

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + str(tree))
        return
    print(indent + str(tree["feature"]))
    for value, subtree in tree["branches"].items():
        print(indent + "  [" + str(value) + "]")
        print_tree(subtree, indent + "    ")


data = [
    {"Customer":"New", "Promo":"Yes", "Visits":"Frequent",
     "Purchase":"Yes"},
```

```
    {"Customer":"New", "Promo":"No", "Visits":"Rare",
     "Purchase":"No"},
    {"Customer":"Returning", "Promo":"Yes", "Visits":"Frequent",
     "Purchase":"Yes"},
    {"Customer":"Returning", "Promo":"No", "Visits":"Frequent",
     "Purchase":"Yes"},
    {"Customer":"New", "Promo":"No", "Visits":"Frequent",
     "Purchase":"Yes"},
    {"Customer":"Returning", "Promo":"No", "Visits":"Rare",
     "Purchase":"No"},
    {"Customer":"New", "Promo":"Yes", "Visits":"Rare",
     "Purchase":"Yes"},
    {"Customer":"Returning", "Promo":"Yes", "Visits":"Rare",
     "Purchase":"Yes"},
    {"Customer":"New", "Promo":"No", "Visits":"Rare",
     "Purchase":"No"},
    {"Customer":"Returning", "Promo":"No", "Visits":"Frequent",
     "Purchase":"Yes"},
]
target = "Purchase"
sample = {"Customer":"New", "Promo":"No", "Visits":"Frequent"}

features = [key for key in data[0] if key != target]
tree = build_tree(data, features, target)

print("Decision Tree:")
print_tree(tree)
print("New sample:", sample)
print("Predicted class:", predict(tree, sample))
```

SAMPLE INPUT

The training dataset and test sample are included in the code above.

SAMPLE OUTPUT

```
Decision Tree:
Visits
  [Frequent]
    -> Yes
  [Rare]
    Promo
      [No]
        -> No
      [Yes]
        -> Yes
New sample: {'Customer': 'New', 'Promo': 'No', 'Visits': 'Frequent'}
Predicted class: Yes
```

RESULT

The ID3 algorithm builds a purchase decision tree and predicts Yes for the supplied sample.